MoverMate: архитектура кода

Сервис «переезд без хаоса»: от решения о переезде до «обжились» — калькулятор и бюджет, план задач с ИИ, учёт коробок и вещей, опись по фото, QR-этикетки, поиск, контроль погрузки и распаковки.

Хакатон PROD, кейс №1, команда 20. Документ описывает то, что реально написано и запущено на вечер 17 октября 2026 (сверено с кодом `master` и живым сайтом после слияния MR 41). Что не сделано, отмечено отдельно (раздел 13).

65	15	219 + 202	98%
операций API	таблиц в базе	тестов сервера и сайта	покрытие сервера

1. Что делает система и какие у неё ограничения

Пользователь создаёт проект переезда (название, дата, адреса необязательны), составляет план задач на три этапа (до, в день, после; стандартный план из 14 задач со сроками создаётся сразу, ИИ-план — по желанию), заводит коробки с комнатой назначения, быстро вписывает содержимое, печатает этикетки с QR-кодом, отмечает погрузку, доставку и распаковку коробок, ведёт статус каждой вещи (упакована, распакована, найдена, потеряна, повреждена) и ищет вещи по названию. Если вещь потерялась, система подсказывает, в каких нераспакованных коробках её искать. Помощники присоединяются к проекту по ссылке-приглашению.

Требование ТЗ	Как выполнено
Простота запуска, без внешних сервисов	Один Docker-образ, SQLite в томе; QR строится в браузере; внешний вызов только к ИИ (LLM-прокси Spirit) и только по согласию
Номера коробок уникальны, повторный скан не создаёт дубль	Ограничение БД `UNIQUE(project_id, number)`, `GET` карточки ничего не создаёт
Статусы только по допустимым переходам	Таблица переходов в сервисе: только следующий этап, прыжок и тот же статус — 409; исправление назад — отдельно, с причиной и записью в журнал
Одинаковые запросы дают одинаковые результаты	Поиск и списки с вторичными ключами сортировки (номер коробки, id)
Данные переживают обновление страницы	Данные в БД; список проектов приходит с сервера после входа, выбранный проект запомнен в браузере
Персональные данные необязательны, нет скрытых передач	Аккаунт — только логин и хэш пароля (scrypt), без почты и телефона; адреса и ответственный необязательны; внешний вызов только к ИИ (LLM-прокси Spirit) по явному согласию, фото не хранится
Сотни коробок и тысячи вещей, пагинация	Индексы, `page` / `page_size`, нагрузочный проект на 300 коробок и 3000 вещей
Проект закрыт для посторонних	Вход по логину и паролю, проект видят только участники; помощники — по ссылке-приглашению; чужой проект отвечает 404

Журнал событий и проверка работоспособности	Таблица `events`, `GET /api/health` с запросом к БД; JSON-логи в формате Sage: каждый запрос, каждая ошибка с причиной и `request_id`, аудит ключевых действий; аппендер отправляет их в Sage
Задеплоено, CI/CD	Пайплайн GitLab: линтеры, тесты, образ, автодеплой на ВМ и проверка живого сайта после каждого слияния в `master`

2. Общая схема

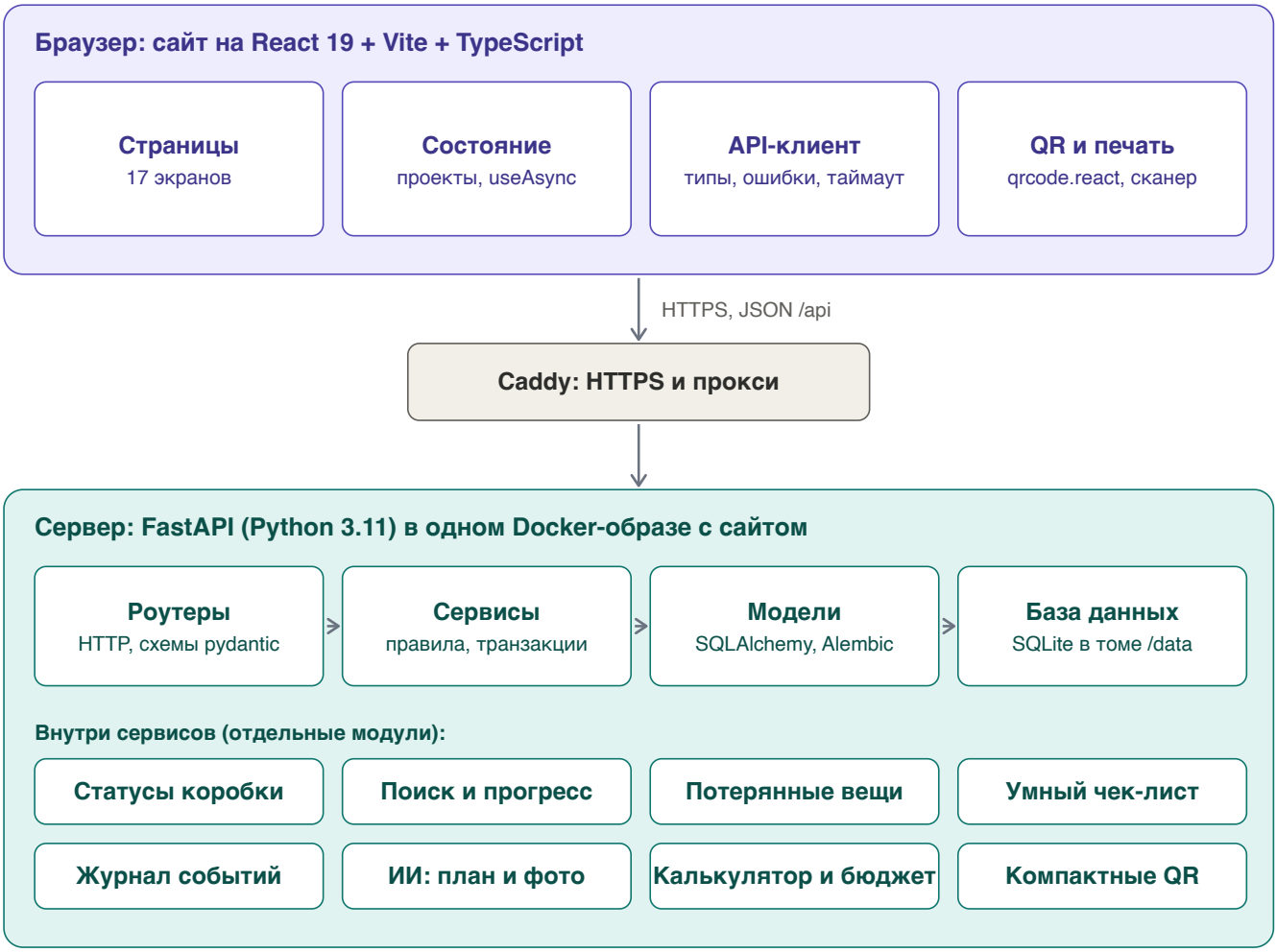


Рис. 1. Слои системы. Сайт и сервер едут в одном образе, база лежит в томе.

Сайт и API работают на одном origin: Caddy отдаёт сайт и проксирует `/api` на FastAPI, поэтому CORS не нужен. В режиме разработки запросы `/api` проксирует Vite (порт 5173 на порт 8000).

3. Сервер (backend)

Три слоя с чёткими обязанностями: роутер принимает и проверяет запрос, сервис содержит все правила и работает с транзакцией, модель описывает таблицы. Бизнес-правил в роутерах нет, поэтому сервисы можно тестировать без HTTP.

Модуль	Что внутри

`main.py`	Создание приложения, прослойка журнала запросов (request_id, время), обработчики ошибок, подключение роутеров, запуск справочников при старте
`routers/`	`auth` (вход, регистрация, демо, пароль, приглашения, участники), `projects` (проекты, задачи, события), `boxes` (коробки, вещи, поиск, этикетки, прогресс, первый день, потерянные, напоминания), `assistant` (умный чек-лист), `ai_photo` (опись и раскладка по фото), `journey` (калькулятор, бюджет), `qr_codes` (компактный QR), `reference` (шаблоны, памятки), `health` (проверка, `/api/config`), общий `confirm_param` для удаления
`services/boxes.py`	Самый крупный модуль (472 строки): выдача номера, машина статусов коробки, массовые переходы, вещи и их статусы, поиск и ранжирование, этикетки, прогресс
`services/auth.py`, `deps.py`	Пароли (scrypt), сессии и приглашения (в базе только SHA-256 токенов), ограничение неудачных входов; проверка участия в проекте в каждом запросе
`services/extras.py`	Чек-лист первого дня, потерянные вещи и подсказки, напоминания
`services/tasks.py`, `projects.py`	Задачи с сортировкой по этапу и сроку; создание проекта со стандартным планом (сроки от даты переезда) или пустым, базовый список первого дня
`services/assistant.py`	Умный чек-лист на локальных правилах, идемпотентное добавление
`services/planner.py`, `vision.py`, `ai_budget.py`	ИИ: запрос к LLM-прокси Spirit с моделью и ключом сервера, вопросы и черновик плана, опись и раскладка по фото, проверка изображения, лимиты попыток в БД
`services/calculator.py`, `budget.py`, `qr_codes.py`	Калькулятор переезда (коэффициенты в начале файла), бюджет «план–факт», постоянные компактные QR-коды
`logging_setup.py`, `sage_appender.py`, `tracing.py`	JSON-логи с `trace_id`, фоновая отправка в Sage, трейсы OpenTelemetry
`services/events.py`	Запись события в той же транзакции, что и изменение
`models.py, db.py`	15 таблиц SQLAlchemy; движок с включёнными внешними ключами для SQLite
`schemas.py`	Контракт API на pydantic: ограничения длины, обрезка пробелов, пустая строка как «не заполнено»
`reference_data.py`, `seed.py`	Шаблоны задач, памятки, чек-лист первого дня, правила умного чек-листа; демо-проект и нагрузочный проект
`static_site.py`	Отдаёт собранный сайт и маршруты SPA (включая ссылки из QR), защищён от `../`; `/api/…` без маршрута остаётся честным 404

Пример: смена статуса коробки



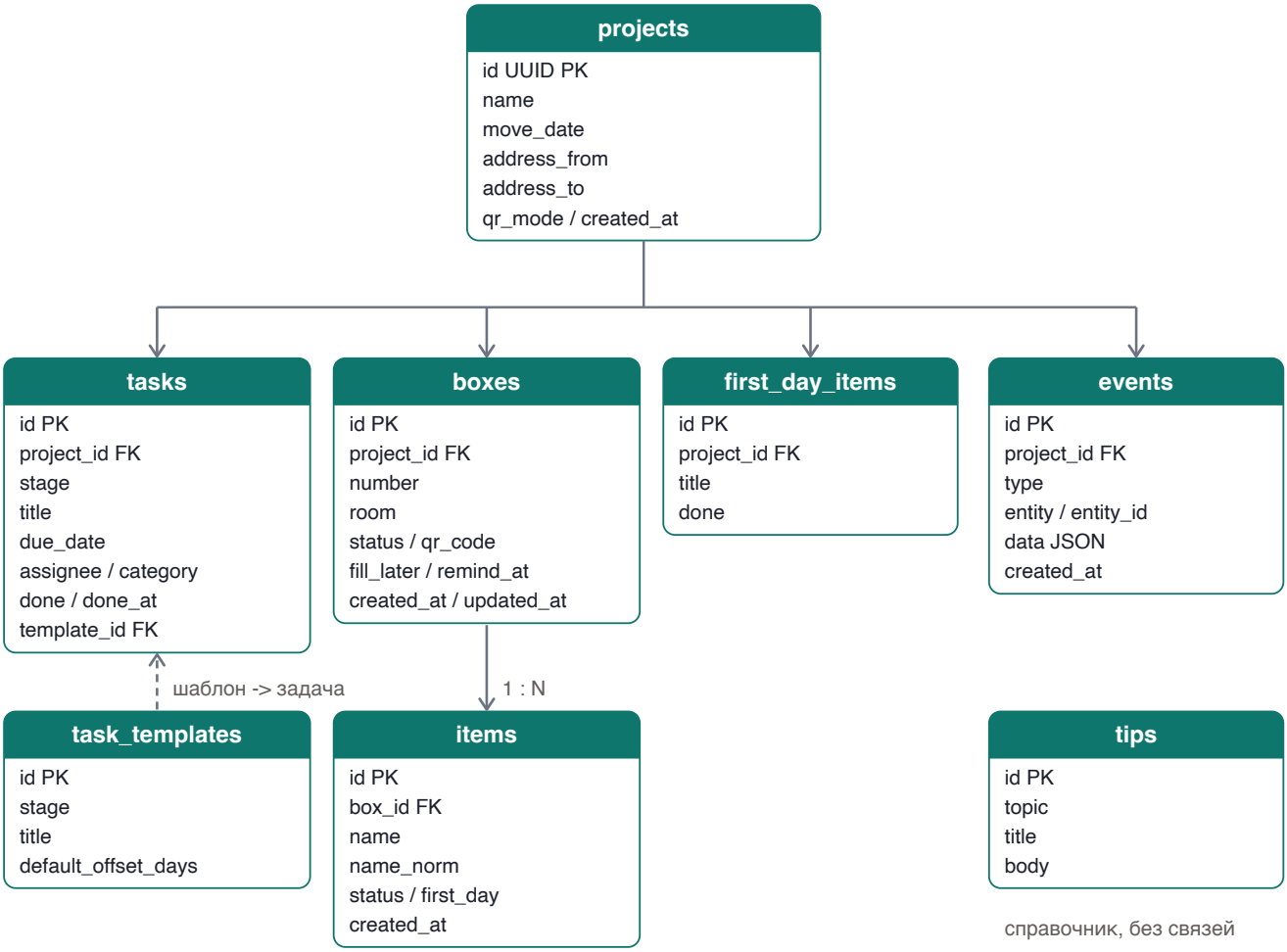
Рис. 2. Один запрос сквозь слои. Блокировка и запись события в одной транзакции.

Единый формат ошибок

```
{"detail": "Недопустимый переход. Проходите этапы по порядку; ошибочную отметку исправляйте
отдельно.",
 "code": "conflict", "request_id": "8a3232f8d2b9"}
```

Коды: 401 не вошли, 403 действие только для владельца, 404 нет сущности или проект чужой, 409 недопустимый переход, 429 перебор пароля, 422 неверные данные или нет `confirm=true` при удалении (с перечнем полей в `errors`), 500 внутренняя ошибка (стек в логе, клиенту только `request_id`). Логи в формате JSON с идентификатором запроса.

4. Модель данных



Все дочерние таблицы удаляются каскадом вместе с проектом (ON DELETE CASCADE).

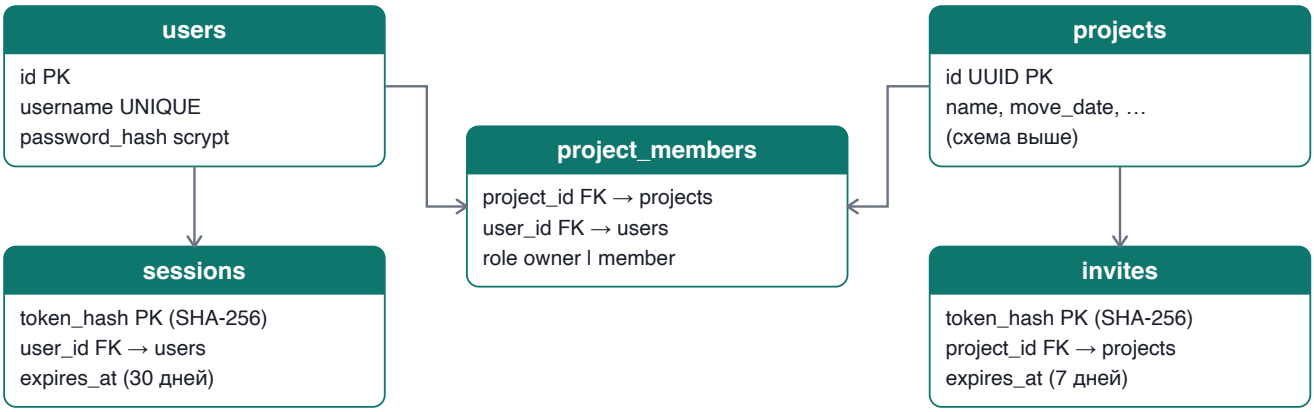
Ограничения: UNIQUE(project_id, number) у boxes; UNIQUE(project_id, template_id) у tasks (шаблон не дублируется).

Рис. 3. Таблицы и связи. Миграции: `backend/alembic/versions/` — `0001` схема, `0002` время и категория задач, `0003` пользователи и доступ, `0004` статус вещи, лимиты ИИ и компактные QR, `0005` пометка «вещь первого дня» и слияние веток, `0007` выравнивание локальных баз, `0008` бюджет `expenses`. Кроме таблиц на рисунке есть `expenses` (бюджет), `box_qr_codes` (постоянные компактные QR) и `ai_usage_budgets` (счётчики лимитов ИИ) — 15 таблиц всего.

- **Номер коробки** хранится отдельным числом, а не как `id`. Сервер выдаёт `max(number)+1` внутри проекта под блокировкой строки проекта; при гонке запись повторяется до трёх раз, а `UNIQUE` не даёт создать дубль. Ограничение: номер удалённой последней коробки будет выдан снова.
- **Статус вещи** — строка `packed | unpacked | found | lost | damaged`; в ответе API для совместимости есть и `lost = (status == "lost")`. Миграция `0004` перенесла старый флажок «потеряна» в статус и откатывается обратно.
- `name_norm` в вещах: имя без регистра, `ё = е`, схлопнутые пробелы. Поиск не зависит от функций СУБД и одинаково работает на SQLite и PostgreSQL.
- **Комната** это строка в коробке: у комнаты нет своих атрибутов, прогресс по комнатам считается группировкой.
- **Индексы:** `boxes(project_id, room, status)`, `items(box_id)`, `items(name_norm)`, `tasks(project_id, stage)`, `events(project_id, id)`, `project_members(project_id, user_id)` (уникальный, проверка доступа в каждом запросе).

- **Справочники** (`task_templates`, `tips`) сидятся при каждом старте и добавляют только недостающее.

Доступ: пользователи, сессии, участники, приглашения



Стрелка — «один ко многим». В базе нет ни одного открытого пароля или токена: утечка БД не даёт войти и присоединиться к проекту.

5. Доменная логика: статусы коробки и вещи



Рис. 4. Машина статусов. Таблица переходов: `TRANSITIONS` в `services/boxes.py`.

Проверка переходов вынесена в чистую функцию `check_transition` и покрыта тестом на все пары статусов: разрешён только следующий этап. Ошибочную отметку исправляет отдельная ручка `status-corrections`: только назад, с причиной, событие `box.status_corrected` с автором. Смена статуса, запись в журнал и проверки идут в одной транзакции со `SELECT … FOR UPDATE` по коробке: две одновременные отметки идут по очереди. Массовая отметка комнаты (`bulk-transitions`) атомарна: одна недопустимая коробка отклоняет всю операцию.

На сайте статус меняется в окне «Сменить статус»: показаны все пять статусов с пояснением, доступен следующий этап; недоступные объясняют, что отметить сначала.

Статус вещи

У каждой вещи свой статус: **упакована** (по умолчанию), **распакована**, **найдена**, **потеряна**, **повреждена**. В отличие от коробки, переходы свободные: вещь может найтись или оказаться повреждённой в любой момент. Статус выбирается в окне по клику на цветную метку вещи. Каждая смена пишется в журнал

(`item.lost`, `item.found` или `item.status_changed` с «из» и «в»); для потерянных вещей работают подсказки, где искать.

Правила, на которых держится сервис

Область	Правило
Поиск	Совпадения ранжируются: точное, с начала слова, вхождение; затем номер коробки и id. `%` и `_` в запросе обычные символы
Прогресс	«Погружено» в комнате = погружена + доставлена + распакована; «Распаковано» = распакованные / все коробки комнаты; охват описью = коробки с вещами / все
Напоминания	Коробка «заполнить позже», без вещей, срок не задан или наступил; вычисляются при запросе, без планировщика
Подсказки о потерянной вещи	Только нераспакованные коробки: где записана (1,0), где лежит похожее по основам слов (0,8), та же комната (0,6); не больше 10, порядок стабилен
Шаблоны задач	Новый проект по умолчанию получает стандартный план — 14 задач трёх этапов со сроками от даты переезда (пустой план — явный выбор); персональный ИИ-план добавляется без дублей
Умный чек-лист	Правила для флагов «дети», «животные», «другой город», «аренда», «документы», «разбор вещей», «обжиться» в `reference_data.py`; добавляются только отсутствующие по названию; работает без внешних вызовов
Удаление	Всегда с `confirm=true`, иначе 422: случайный запрос не удаляет данные

6. API

Все 65 операций на 50 путях описаны в Swagger (`/api/docs`) и в `docs/API.md`. Вход по логину и паролю: сессия в httpOnly-cookie, проект видят только его участники (владелец и приглашённые по ссылке), чужой проект отвечает 404. Длинные списки идут страницами (`page`, `page_size` до 200).

Метод	Путь	Назначение
Служебные (4)		
GET	`/api/health`	Проверка работоспособности с запросом к БД
GET	`/api/config`	Ключ и провайдер подсказок адреса (MapTiler или Google), если заданы
GET	`/api/task-templates`	Шаблоны задач
GET	`/api/tips`	Памятки (фильтр topic)
Вход (6)		
POST	`/api/auth/register`	Регистрация: логин и пароль, сразу вход
POST	`/api/auth/login`	Вход (429 при переборе)
POST	`/api/auth/demo`	Демо-вход для жюри без пароля
POST	`/api/auth/logout`	Выход
POST	`/api/auth/password`	Смена пароля, остальные входы закрываются
GET	`/api/auth/me`	Кто вошёл
Приглашения и участники (6)		
POST	`/api/projects/{pid}/invites`	Ссылка-приглашение на 7 дней (владелец)
DELETE	`/api/projects/{pid}/invites`	Отозвать все ссылки (владелец)
GET	`/api/invites/{token}`	Куда зовут (без входа)
POST	`/api/invites/{token}/accept`	Присоединиться к проекту
GET	`/api/projects/{pid}/members`	Участники проекта
DELETE	`/api/projects/{pid}/members/{user_id}?confirm=true`	Убрать участника или выйти самому
Проекты (6)		
GET	`/api/projects`	Мои проекты с ролью
POST	`/api/projects`	Создать проект: стандартный или пустой план, базовый список первого дня, проверенный ИИ-план
GET	`/api/projects/{pid}`	Получить проект
PATCH	`/api/projects/{pid}`	Изменить название, дату, адреса
DELETE	`/api/projects/{pid}?confirm=true`	Удалить проект со всем содержимым
GET	`/api/projects/{pid}/events`	Журнал событий (страницы, фильтр type)

Задачи (5)		
GET	`/api/projects/{pid}/tasks`	Задачи (фильтры stage, done)
POST	`/api/projects/{pid}/tasks`	Добавить свою задачу
PATCH	`/api/projects/{pid}/tasks/{tid}`	Изменить или отметить выполнение
DELETE	`/api/projects/{pid}/tasks/{tid}?confirm=true`	Удалить задачу
POST	`/api/projects/{pid}/tasks/apply-templates`	Добавить типовые задачи (вариант «Обычный переезд» в умном чек-листе)
Коробки (9)		
GET	`/api/projects/{pid}/boxes`	Список коробок (room, status, fill_later, number, страницы)
POST	`/api/projects/{pid}/boxes`	Создать коробку, номер выдаёт сервер
GET	`/api/projects/{pid}/boxes/{n}`	Карточка коробки (её открывает QR)
PATCH	`/api/projects/{pid}/boxes/{n}`	Комната, «заполнить позже»
DELETE	`/api/projects/{pid}/boxes/{n}?confirm=true`	Удалить коробку с вещами
POST	`/api/projects/{pid}/boxes/{n}/transitions`	Сменить статус по допустимым переходам
POST	`/api/projects/{pid}/boxes/bulk-transitions`	Массовая смена статуса, всё или ничего
GET	`/api/projects/{pid}/boxes/{n}/label`	Данные этикетки (qr_path)
GET	`/api/projects/{pid}/labels`	Пакет этикеток (numbers=1,2,3, до 100)
Вещи (8)		
GET	`/api/projects/{pid}/boxes/{n}/items`	Вещи в коробке (страницы)
POST	`/api/projects/{pid}/boxes/{n}/items`	Добавить вещь
POST	`/api/projects/{pid}/boxes/{n}/items/bulk`	Быстрый ввод: строка = вещь, до 200
PATCH	`/api/projects/{pid}/items/{id}`	Переименовать, сменить статус вещи
DELETE	`/api/projects/{pid}/items/{id}?confirm=true`	Удалить вещь
GET	`/api/projects/{pid}/items/lost`	Потерянные вещи
GET	`/api/projects/{pid}/items/first-day`	Вещи первого дня
GET	`/api/projects/{pid}/items/{id}/hints`	Где искать потерянную вещь
Поиск и контроль (3)		
GET	`/api/projects/{pid}/search?q=`	Поиск вещи среди всех коробок
GET	`/api/projects/{pid}/progress`	Статусы и прогресс по комнатам
GET	`/api/projects/{pid}/reminders`	Коробки, которые пора описать
Первый день (4)		
GET	`/api/projects/{pid}/first-day`	Чек-лист первого дня
POST	`/api/projects/{pid}/first-day`	Добавить пункт
PATCH	`/api/projects/{pid}/first-day/{id}`	Отметить или переименовать

DELETE	`/api/projects/{pid}/first-day/{id}?confirm=true`	Удалить пункт
Умный чек-лист (2)		
POST	`/api/projects/{pid}/assistant/checklist`	Предложить задачи (дети, животные, другой город, аренда, документы, разбор вещей, «обжиться»)
POST	`/api/projects/{pid}/assistant/checklist/apply`	Добавить предложенное, идемпотентно
Коробки: исправление и компактный QR (2)		
POST	`/api/projects/{pid}/boxes/{n}/status-corrections`	Вернуть на более ранний этап с причиной
POST	`/api/boxes/resolve-code`	Компактный QR MM1:… → проект и номер коробки (только участнику)
Первый день: в коробку (1)		
POST	`/api/projects/{pid}/first-day/{item_id}/bind`	Положить пункт первого дня в коробку
ИИ по согласию (4)		
POST	`/api/projects/plan-questions`	Уточняющие вопросы к плану (до 3)
POST	`/api/projects/plan-preview`	Черновик плана с ответами
POST	`/api/projects/{pid}/boxes/{n}/photo-inventory`	Опись коробки по фото
POST	`/api/projects/{pid}/room-plan`	Раскладка комнаты по коробкам по фото
Калькулятор и бюджет (5)		
POST	`/api/calculator`	Оценка переезда (после входа)
GET	`/api/projects/{pid}/expenses`	Бюджет: статьи, план, факт, отклонение
POST	`/api/projects/{pid}/expenses`	Добавить статью
PATCH	`/api/projects/{pid}/expenses/{id}`	Изменить статью или внести факт
DELETE	`/api/projects/{pid}/expenses/{id}`	Удалить статью (?confirm=true)

7. Сайт (frontend)

React 19, TypeScript, Vite, `react-router-dom`, обычный CSS без UI-библиотек, иконки `lucide-react`, шрифты Manrope и IBM Plex Mono (дизайн Кусмарцева Фёдора: крафтовая палитра, «картонные» коробки). Зависимостей минимум, код короткий. Сканер QR (`html5-qrcode`) подгружается отдельным куском только при открытии экрана.

Маршруты и экраны

Путь	Экран
`/`	План: маршрут и счётчики, вкладки трёх этапов (новая задача — на открытый этап) и «Первый день» («Что подготовить» и «В коробках»); отметки срабатывают сразу
`/boxes`	Коробки: поиск вещи и номера, комнаты, напоминания
`/projects/:pid/boxes/:n`	Карточка (сюда ведёт QR)
… `/label`	Этикетка: номер, комната, QR
`/labels`, `/labels/print`	Список и печать всех этикеток
`/progress`	Итог переезда, счётчики, комнаты с чек-листом погрузки, «Первый день», потерянные вещи
`/scan`, `/new`	Сканер, новый проект
`/invite/:token`	Приглашение в проект
`/journey`	Путь переезда: 7 этапов со статусом по данным проекта
`/calculator`, `/budget`	Калькулятор переезда и бюджет «план–факт» (после входа)
`/tips`, `/about`	Памятка (ссылка внизу «Коробок»), «О нас»
`/profile`	Личный кабинет: «Выбрать проект» (карточки проектов), настройки проекта (адреса, участники, приглашение), смена пароля, выход

Слои кода

Папка	Роль
`pages/`	Экраны, по одному файлу
`components/`	Каркас (`Shell`: сворачиваемое меню, шапка с разделом, сканером, уведомлениями и профилем), окна (`Modal`, выбор статуса, уведомления), этикетка, формы
`state/`	Вход (`AuthContext`), проекты с сервера, `useAsync`, `useAction`, `useDebounced`
`api/`	Типы по контракту, клиент, функции эндпоинтов
`ui/`	Форматы, `StateView`, бейдж статуса

без входа	«О нас», «Путь переезда», «Вопросы» (частые вопросы), вход и регистрация (пароль дважды); остальные разделы просят войти

Как работают экраны

- Каждый экран грузит данные через `useAsync` и показывает единые состояния «загрузка / ошибка с повтором / пусто» (`StateView`). Действия пользователя ловят ошибки сервера и показывают понятный текст.
- Клиент API (`api/client.ts`) ходит на `/api` того же origin, ставит таймаут 20 секунд, разбирает формат ошибок сервера и превращает HTML-ответ прокси (502) в понятное сообщение.
- Без входа открыты «О нас», «Путь переезда», «Вопросы» и вход; любой другой адрес (коробка по QR, приглашение) показывает «О нас» с предложением войти, а адрес не меняется: после входа открывается та же страница. Список проектов приходит с сервера; в `localStorage` запомнен только выбранный проект, после выхода список стирается.
- **QR:** рисуется в браузере (`qrcode.react`). Универсальный формат (по умолчанию) кодирует ссылку `{origin}/projects/{pid}/boxes/{n}` — её открывает любая камера телефона; компактный `MM1:…` (20 символов, мельче матрица) читает встроенный сканер через `/api/boxes/resolve-code`. Сканер принимает оба формата, формат выбирает владелец проекта.
- **Печать:** четыре шаблона (стандарт, только QR, с заметками, без QR), выбор коробок и экземпляров, A4 на 4/8/12 этикеток с предпросмотром; `window.print()` и `@media print` оставляют только листы, системный диалог печатает или сохраняет в PDF.
- **Уведомления:** колокольчик в шапке показывает напоминания об описи коробок и последние события журнала понятным текстом.
- **Адаптивность:** на телефоне вкладки снизу, на ноутбуке бежевая боковая панель (сворачивается; «Путь переезда» отдельным пунктом вверху); цвет каждого статуса один на всех экранах.

8. ИИ, путь переезда и наблюдаемость

Всё, что выходит за минимум ТЗ, держится на тех же слоях (роутер → сервис → модель) и тех же правилах доступа: только участники проекта, данные меняются только явным действием пользователя.

Возможность	Как устроено
ИИ-план с уточняющими вопросами	`POST /api/projects/plan-questions` → до 3 вопросов с вариантами; ответы уходят в `plan-preview`; черновик сохраняется только при создании проекта. Модель — `AI_MODEL` (Claude Haiku 5.5 через LLM-прокси Spirit)
Опись коробки и раскладка комнаты по фото	`photo-inventory` и `room-plan`: фото уменьшается в браузере до 1280 px (без EXIF), проверяется на сервере до списания лимита, модели уходит один раз и не хранится; ответ проверяется схемой и только предлагается. Модель — `AI_VISION_MODEL` (Claude Sonnet 5.5) со своим ключом `AI_VISION_API_KEY`
Защита ИИ	явное согласие (`consent: true`, иначе 422); лимиты в БД — 10 запросов в час на пользователя и 50 в день на всех, резерв попытки до платного вызова; ключи только на сервере; ошибки провайдера наружу без тела и ключа
Калькулятор переезда	`POST /api/calculator` (после входа): коробки, объём, машина, грузчики, часы, бюджет «от–до» по статьям, срок подготовки, советы по дате; правила и коэффициенты в `services/calculator.py`
Бюджет «план–факт»	таблица `expenses` (миграция 0008): статьи с планом и фактом, отклонение по оплаченному; смета калькулятора переносится одной кнопкой
Путь переезда	страница `/journey`: 7 этапов (решение, новое жильё, подготовка, подрядчики и бюджет, упаковка, день переезда, обжиться), статус каждого считается по данным проекта; пункт в самом верху меню
Логи и трейсы	JSON-логи в Sage: в каждой строке `trace_id` / `span_id` и текст `message`, у ошибок `error_type`; OpenTelemetry: спан запроса по шаблону маршрута, спаны SQL без значений параметров, спан вызова ИИ; ответ несёт `x-trace-id`

9. Ключевые решения

Решение	Почему
Адаптивный сайт и Android-оболочка вместо нативного приложения	Срок три дня: один код для телефона и ноутбука, QR открывается любой камерой, жюри проверяет без установки. APK (`mobile/`, Capacitor) открывает тот же сайт — те же API, вход и данные, без второй реализации.
Вход по логину и паролю, участники и приглашения	Проект закрыт для посторонних, помощники приходят по ссылке на 7 дней. Храним только логин и хэш пароля (scrypt), токены сессий и приглашений — как SHA-256
Три слоя: роутер, сервис, модель	Правила не размазаны по контроллерам; сервисы тестируются без HTTP
Таблица переходов статусов	Новый статус или правило это одна строка плюс тест

Статусы коробки строго по порядку, исправление — отдельно с причиной	П. 8 ТЗ требует допустимых переходов; случайная отметка не перескакивает этапы, а ошибка исправляется явно и остаётся в журнале (решение Кусмарцева Фёдора 17.10). Статус вещи свободный: вещь может найтись или повредиться в любой момент
Выбор статуса во всплывающем окне	Все варианты видны сразу с пояснением; одно окно для коробки и вещи; комнату целиком отмечают в чек-листе погрузки на «Прогрессе» (атомарно)
Стандартный план со сроками от даты переезда	сразу видно, что и когда делать; пустой план — явный выбор при создании (решение Кусмарцева Фёдора 17.10)
Номер в транзакции под блокировкой + UNIQUE	Уникальность гарантирует база, даже если сервис ошибётся
Один образ с сайтом и API	Проще деплой и шаблон CI организаторов (`Dockerfile` в корне); SQLite в томе: мало движущихся частей
Автодеплой по шаблону организаторов	Каждое слияние в `master` проверено и выкачено одинаково; перед выкаткой копия базы, после — проверка живого сайта
Caddy с автоматическим HTTPS	Камера сканера работает только по HTTPS; сертификат получается и продлевается сам
Умный чек-лист на правилах, без внешнего ИИ	ТЗ запрещает скрытые передачи данных; результат предсказуем и проверяем тестами

10. Развёртывание

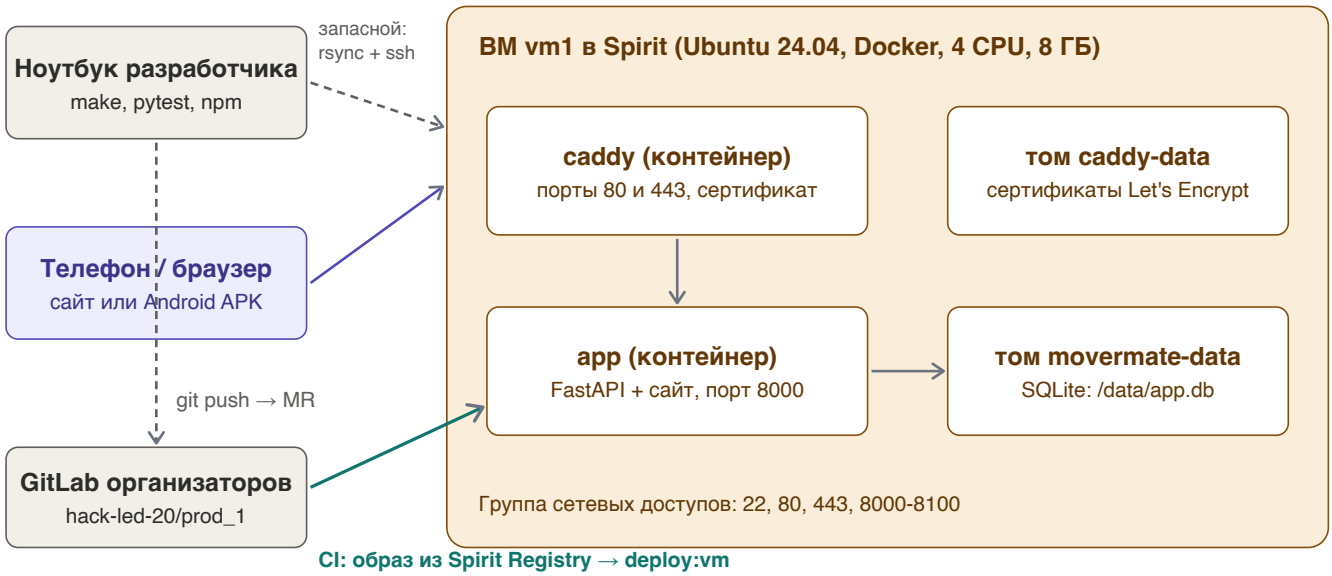
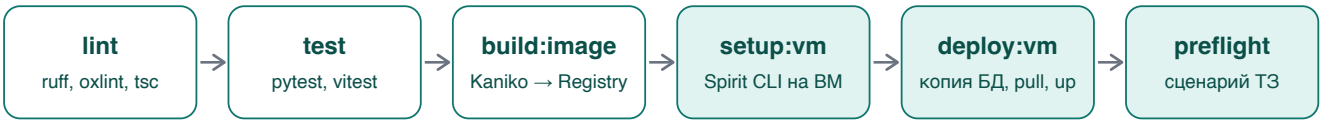


Рис. 5. Окружение. Основной путь — пайплайн GitLab; ручная выкатка осталась запасной.

- **Образ:** `Dockerfile` в корне, два этапа: сборка сайта (Node 22) и сервер (Python 3.11). При старте контейнер применяет миграции и запускает `uvicorn`; проверка живости `/api/health`.
- **Compose:** `deploy/docker-compose.yml` поднимает `app` и `caddy`; имя сервера для HTTPS задаётся в `deploy/.env` (`SITE_ADDRESS`).
- **Адрес:** https://hack-led-20-vm1-12902f.vmaas-spirit.t-education.com (сертификат Let's Encrypt).
- **Выкатка:** автоматически из пайплайна после слияния MR в `master` (раздел 10). Запасной путь — `rsync` исходников и `docker compose up -d --build` (`docs/OPERATIONS.md`).

- **Локально:** `scripts/run-site.sh` (сервер отдаёт API и сайт на порту 8000) или `make dev` + `npm run dev`.

11. CI/CD



белые — на каждой ветке и MR; зелёные — только после слияния в master

Рис. 6. Пайплайн `.gitlab-ci.yml` по шаблону организаторов `templates/l2sh-template`.

- **Процесс:** `master` защищён; изменения идут веткой и Merge Request. Пайплайн MR должен быть зелёным, после слияния пайплайн `master` сам выкатывает сайт.
- **Тесты в CI:** сервер на SQLite с отчётом покрытия (cobertura, процент виден в GitLab) и те же тесты на PostgreSQL — там проверяется параллельная выдача номеров коробок; миграции вверх-вниз-вверх; сайт — vitest с покрытием и сборка; результаты тестов обеих частей видны во вкладке Tests (junit).
- **Выкатка:** образ `registry.spirit.t-education.com/hack-led-20/prod_1:<sha>`; на ВМ копия базы в `/root/backups`, `docker compose pull app` и `up -d` в существующем стеке с Caddy, поэтому адрес и сертификат не меняются; база и сертификаты не трогаются, в `deploy/.env` записывается только `APP_IMAGE` (ручной перезапуск не поднимет старый образ).
- **Проверка после выкатки:** `preflight:prod` проходит весь сценарий ТЗ на живом сайте через демо-вход. Если он красный, пайплайн красный.
- **Отличия от шаблона:** адреса `culab.ru` заменены на `t-education.com` (с раннеров не резолвятся), образ для ssh — `kroniak/ssh-client`, ключ `SSH_PRIVATE_KEY` очищается от `\r`.

12. Тестирование и проверка

Что	Сколько	Что проверяется
Сервер (pytest)	219 тестов, покрытие 98%	Правила статусов (все пары, пустая коробка), статусы вещей и журнал, стандартный план и сроки, самовосстановление демо, номера и дубли, поиск и порядок, прогресс, подсказки, напоминания, массовая погрузка, чек-лист и его идемпотентность, миграция совпадает с моделями, отдача сайта и защита от `../`
Сайт (vitest)	202 теста, покрытие строк 64%	Разбор ссылки из QR, форматы и склонения, клиент API (сеть, 409/422, 502), хранилище проектов (в том числе недоступное), состояния экрана, формы проекта, задачи и входа, окно выбора статуса, личный кабинет
Сквозная проверка	`scripts/e2e.sh`: 32 + 32 + 20 + 36	Минимум ТЗ и доп. функции по API прода, права участников, прокликивание интерфейса в headless Chrome (на проде — без создания аккаунта)
Проверка выкатки	`scripts/preflight.sh`	Сайт и ссылка из QR, health, Swagger, демо-данные, полный сценарий ТЗ на временном проекте; запускается в CI после каждой выкатки

В локальном прогоне на SQLite пропущены 2 теста: параллельная выдача номеров (идёт в CI на PostgreSQL) и проверка аннотаций (только на Python 3.14). Нагрузка (300 коробок, 3000 вещей) замерена на проде: списки, поиск и прогресс отвечают за 15–20 мс.

13. Что не сделано и ограничения

- ИИ зависит от квоты аккаунтов Spirit; без провайдера сайт предлагает стандартный план, коробки заполняются вручную.
- Трейсы в Sage ждут открытия порта коллектора 4319; `trace_id` уже есть в каждой строке лога и в заголовке `x-trace-id`.
- Цены калькулятора — ориентир (коэффициенты в `calculator.py`), не цена подрядчика.
- **База — SQLite на одной ВМ.** Резервные копии: каждый час по cron и перед каждой выкаткой (`deploy/backup.sh`: онлайн-копия, проверка целостности, последние 48 копий). Отказоустойчивости второй ВМ нет.
- **Нет восстановления пароля**: почту не храним. Пароль можно сменить, зная текущий.
- **Номер удалённой последней коробки** выдаётся снова (номер считается как `max + 1`).
- **Напоминания** видны только в интерфейсе, без push и почты (осознанно: ТЗ не требует внешних каналов).
- **Камера сканера** требует HTTPS (на ВМ включён); на реальном телефоне проверена, есть ручной ввод номера.
- **Android-приложение** — оболочка над живым сайтом: без сети показывает экран «Нет связи», офлайн-режима нет.
- **Распознавание рукописного номера** (локальный OCR) — эксперимент, в обычной сборке выключен: качество почерка не проверено.

14. Структура репозитория

```
backend/            сервер: app/ (код), alembic/ (10 миграций), tests/ (219 тестов)
frontend/           сайт: src/ (код, 202 теста)
deploy/             docker-compose.yml, Caddyfile, backup.sh, .env.example, sage/alerts.yaml
.gitlab-ci.yml      пайплайн: lint → test → образ → деплой → preflight
Dockerfile          образ: сайт + сервер
docs/               CASE, ARCHITECTURE, API, FEATURES, OPERATIONS (.md), этот PDF
mobile/             Android-приложение (Capacitor), MoverMate.apk
scripts/            preflight, e2e, run-site, vm-env, ai-setup, sage-setup, gitlab-login (.sh)
```